Week 3 Module

By Mohit Sharma, 2404921540131

Problem-1:

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

- First line contains an integer **N**, representing the number of placed students.
- Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

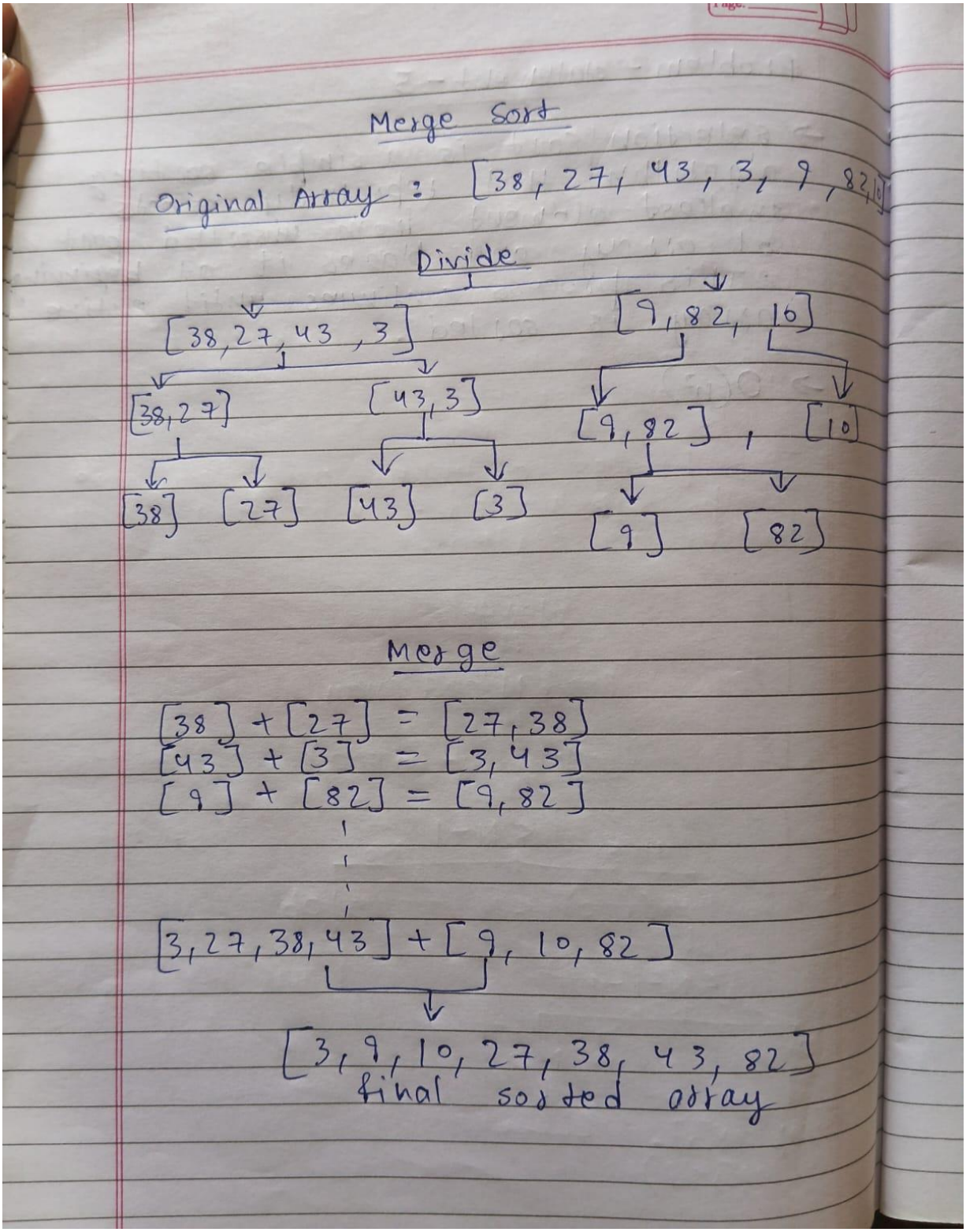
Output

Print the package values in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $100 \leq \text{Package} \leq 10000$

Theory:-


Merge Sort
Original Array : [38, 27, 43, 3, 9, 82, 10]
Divide
[38, 27, 43, 3]
[9, 82, 10]
[38, 27]
[43, 3]
[9, 82]
[10]
[38]
[27]
[43]
[3]
[9]
[82]
Merge
[38] + [27] = [27, 38]
[43] + [3] = [3, 43]
[9] + [82] = [9, 82]
[3, 27, 38, 43] + [9, 10, 82]
[3, 9, 10, 27, 38, 43, 82]
final sorted array

Solution-1:

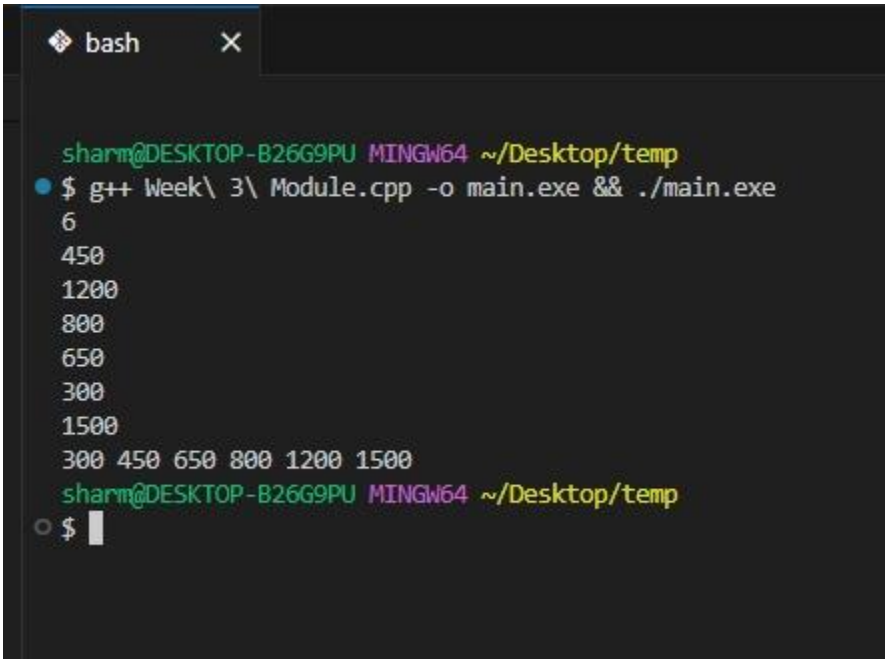
Week 3 Module.cpp

Week 3 Module.cpp > main()

```cpp
#include <iostream>
#include <vector>
using namespace std;

void merge(vector<int> &arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }
    for (int i = 0; i < n2; i++){
        R[i] = arr[mid + 1 + i];
    }
    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```cpp
void mergeSort(vector<int> &arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int N;
    cin >> N;

    vector<int> arr(N);

    for (int i = 0; i < N; i++)
        cin >> arr[i];

    mergeSort(arr, 0, N - 1);

    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";

    return 0;
}
```

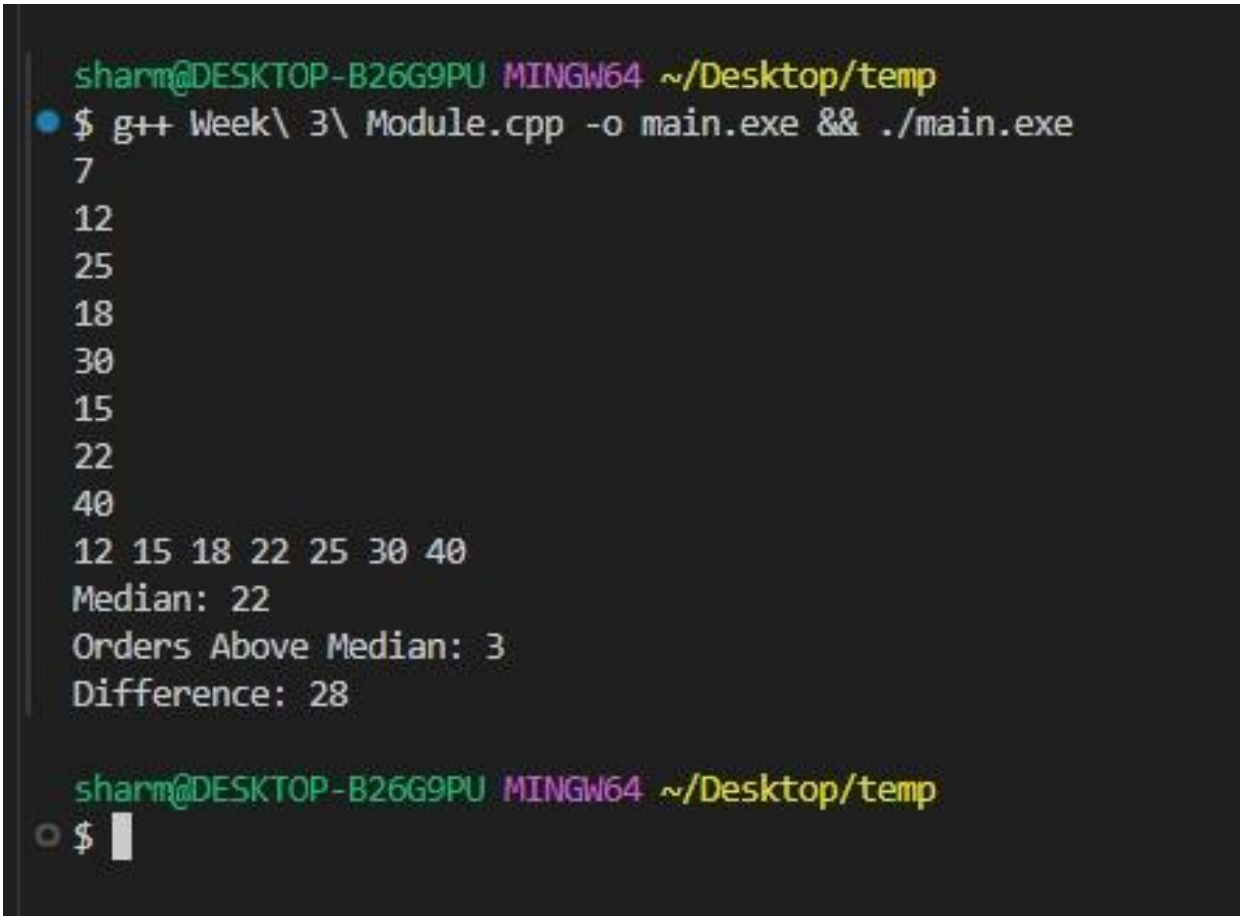
Result-1:



Question-2:

```
54  int main() {
55      int N;
56      cin >> N;
57
58      vector<int> arr(N);
59
60      for (int i = 0; i < N; i++)
61          cin >> arr[i];
62
63      mergeSort(arr, 0, N - 1);
64      for (int i = 0; i < N; i++)
65          cout << arr[i] << " ";
66      cout << endl;
67
68      int median;
69      if (N % 2 == 1)
70          median = arr[N / 2];
71      else
72          median = (arr[N / 2 - 1] + arr[N / 2]) / 2;
73
74      cout << "Median: " << median << endl;
75      int count = 0;
76      for (int i = 0; i < N; i++) {
77          if (arr[i] > median)
78              count++;
79      }
80
81      cout << "Orders Above Median: " << count << endl;
82
83      cout << "Difference: " << arr[N - 1] - arr[0] << endl;
84
85      return 0;
86  }
```

Result-2:


sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 3\ Module.cpp -o main.exe && ./main.exe
7
12
25
18
30
15
22
40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$

Topic-2::

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

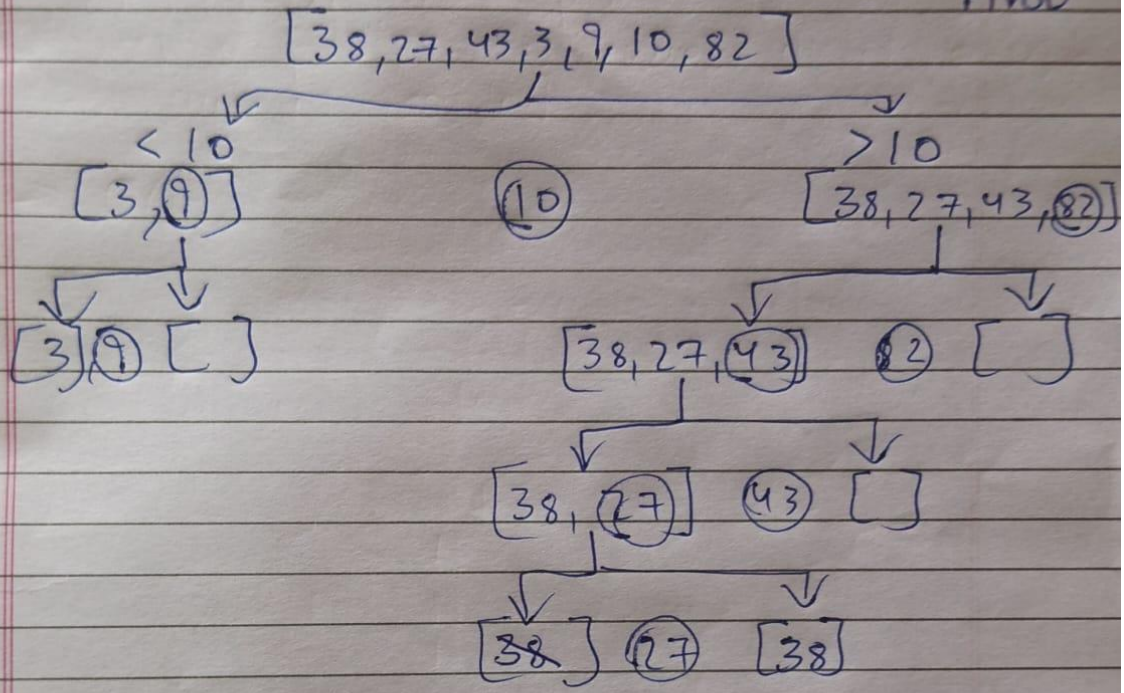
- $1 \leq N \leq 100000$
- $1 \leq$ Response Time ≤ 10000

Flow chart:

Quick Sort

original array : [38, 27, 43, 3, 9, 82, 10]

Pivot



Merge back :->

left : [3, 9]

Mid : [10]

Right : [27, 38, 43, 82]

Final sorted array : [3, 9, 10, 27, 38, 43, 82]

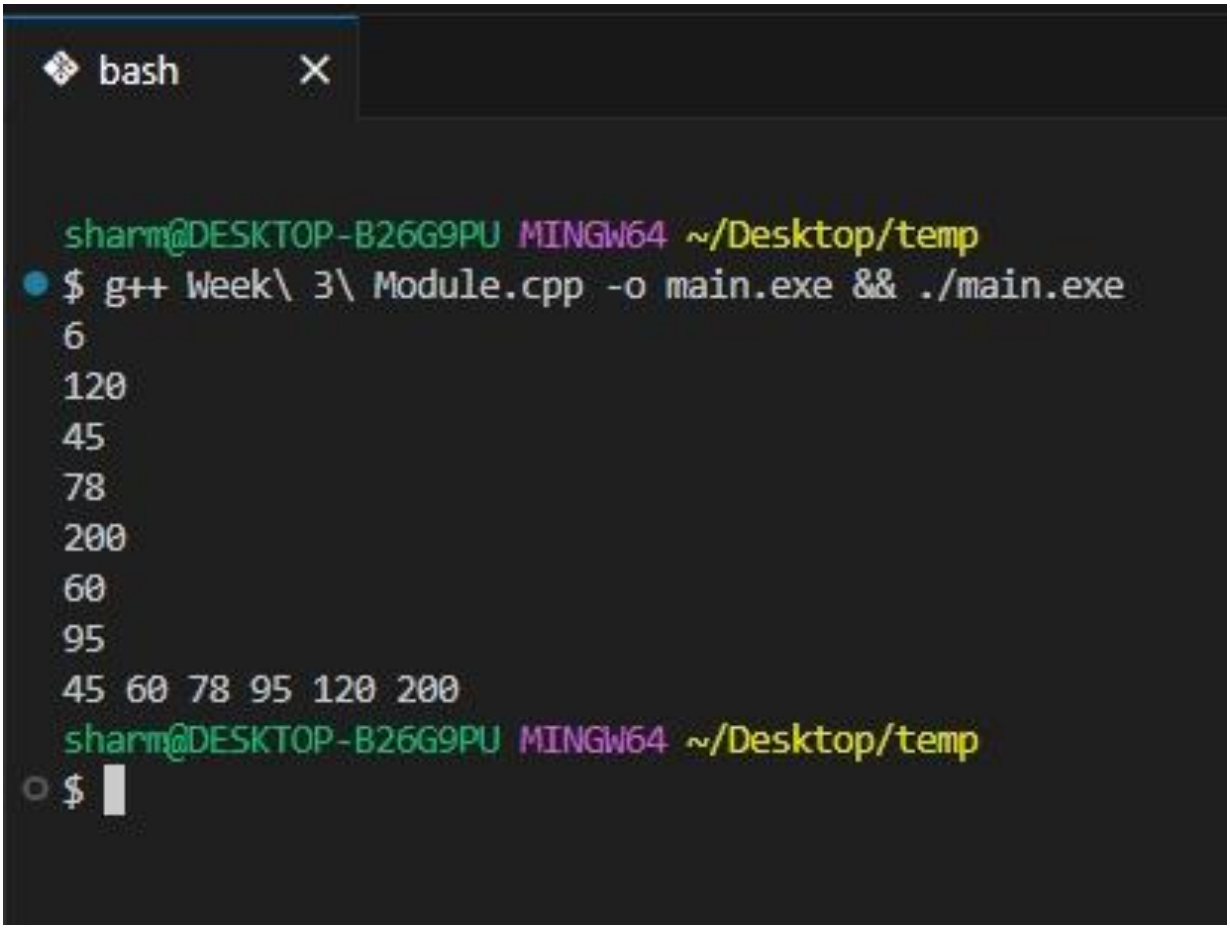
Question-1:

Week 3 Module.cpp

Week 3 Module.cpp > main()

```cpp
#include <iostream>
#include <vector>
using namespace std;

int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int N;
    cin >> N;

    vector<int> arr(N);

    for (int i = 0; i < N; i++)
        cin >> arr[i];

    quickSort(arr, 0, N - 1);

    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Result-1:


bash
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 3\ Module.cpp -o main.exe && ./main.exe
6
120
45
78
200
60
95
45 60 78 95 120 200
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$

Question-2:

Week 3 Module.cpp

Week 3 Module.cpp > main()

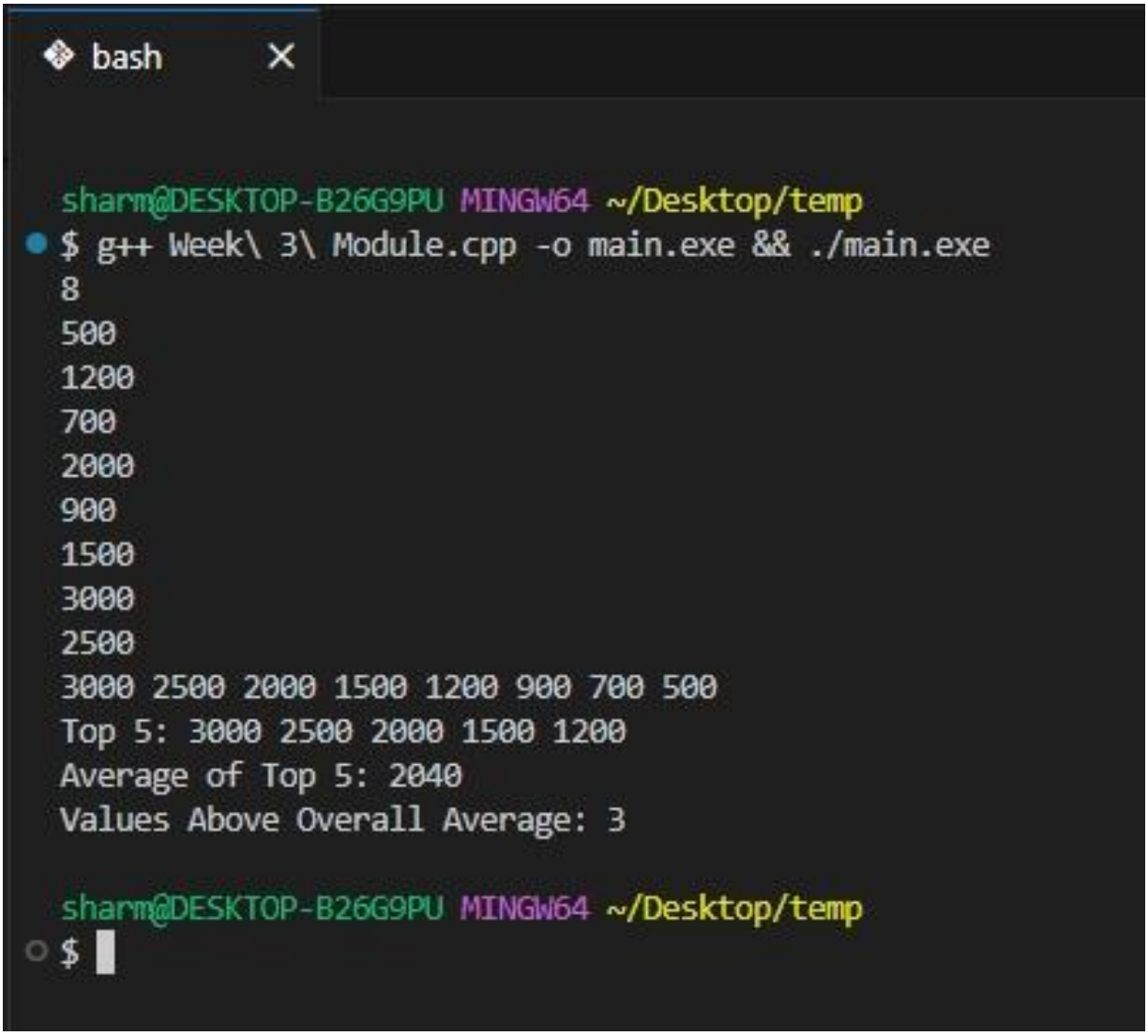
```cpp
#include <iostream>
#include <vector>
using namespace std;

int partition(vector<long long> &arr, int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] > pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<long long> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Week 3 Module.cpp

Week 3 Module.cpp > main()

```cpp
int main() {
    int N;
    cin >> N;

    vector<long long> arr(N);
    long long sum = 0;

    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        sum += arr[i];
    }

    quickSort(arr, 0, N - 1);
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;

    cout << "Top 5: ";
    long long top5Sum = 0;
    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
        top5Sum += arr[i];
    }
    cout << endl;

    cout << "Average of Top 5: " << top5Sum / 5 << endl;

    double overallAverage = (double)sum / N;
    int count = 0;

    for (int i = 0; i < N; i++) {
        if (arr[i] > overallAverage)
            count++;
    }

    cout << "Values Above Overall Average: " << count << endl;

    return 0;
}
```

Result-2:


bash
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 3\ Module.cpp -o main.exe && ./main.exe
8
500
1200
700
2000
900
1500
3000
2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$

Topic-3:

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed O(N log N) performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer **N**.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $0 \leq Score \leq 1000000$

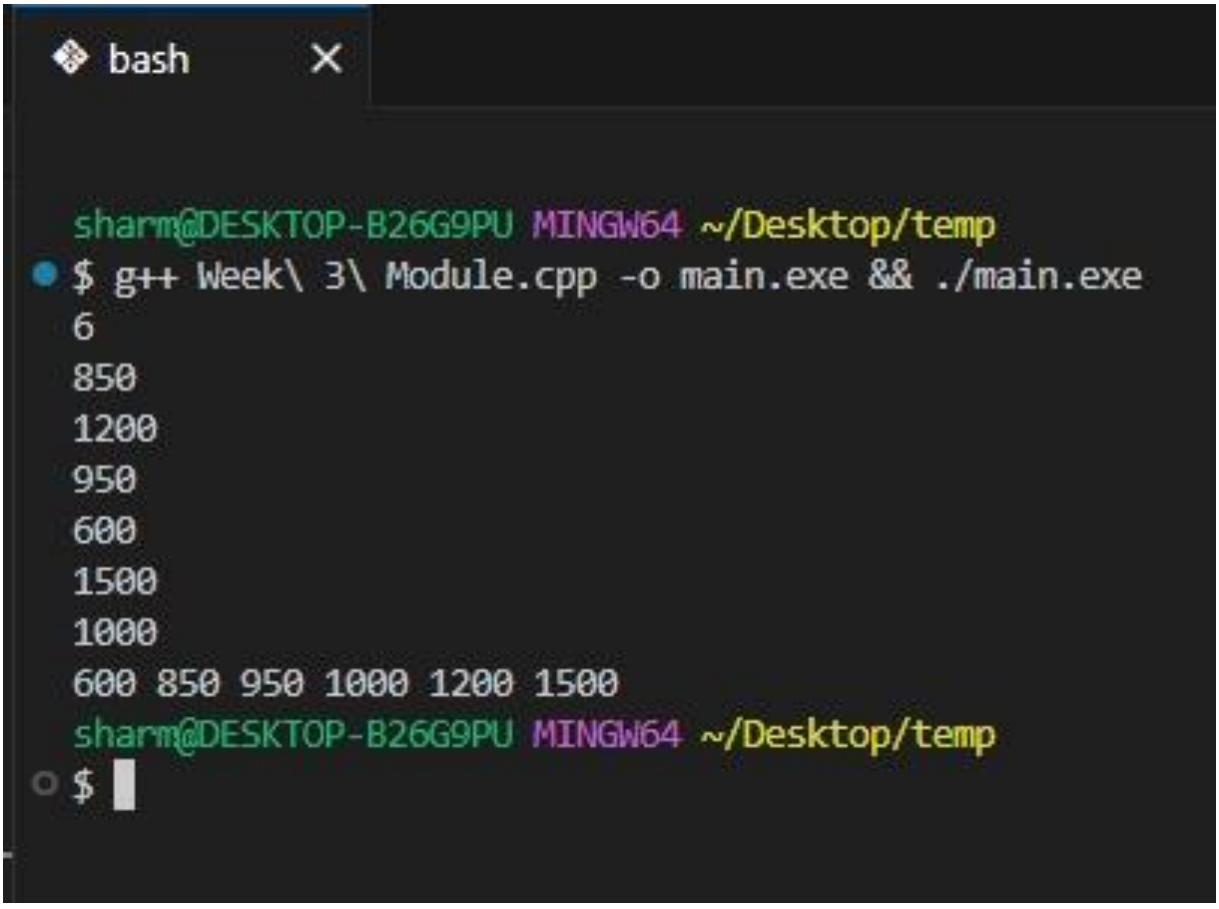
Question-1:

Week 3 Module.cpp

Week 3 Module.cpp > heapSort(vector<int>&, int)

```cpp
#include <iostream>
#include <vector>
using namespace std;

void heapify(vector<int> &arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int> &arr, int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int N;
    cin >> N;

    vector<int> arr(N);

    for (int i = 0; i < N; i++)
        cin >> arr[i];

    heapSort(arr, N);

    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Result-1:


bash
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 3\ Module.cpp -o main.exe && ./main.exe
6
850
1200
950
600
1500
1000
600 850 950 1000 1200 1500
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$

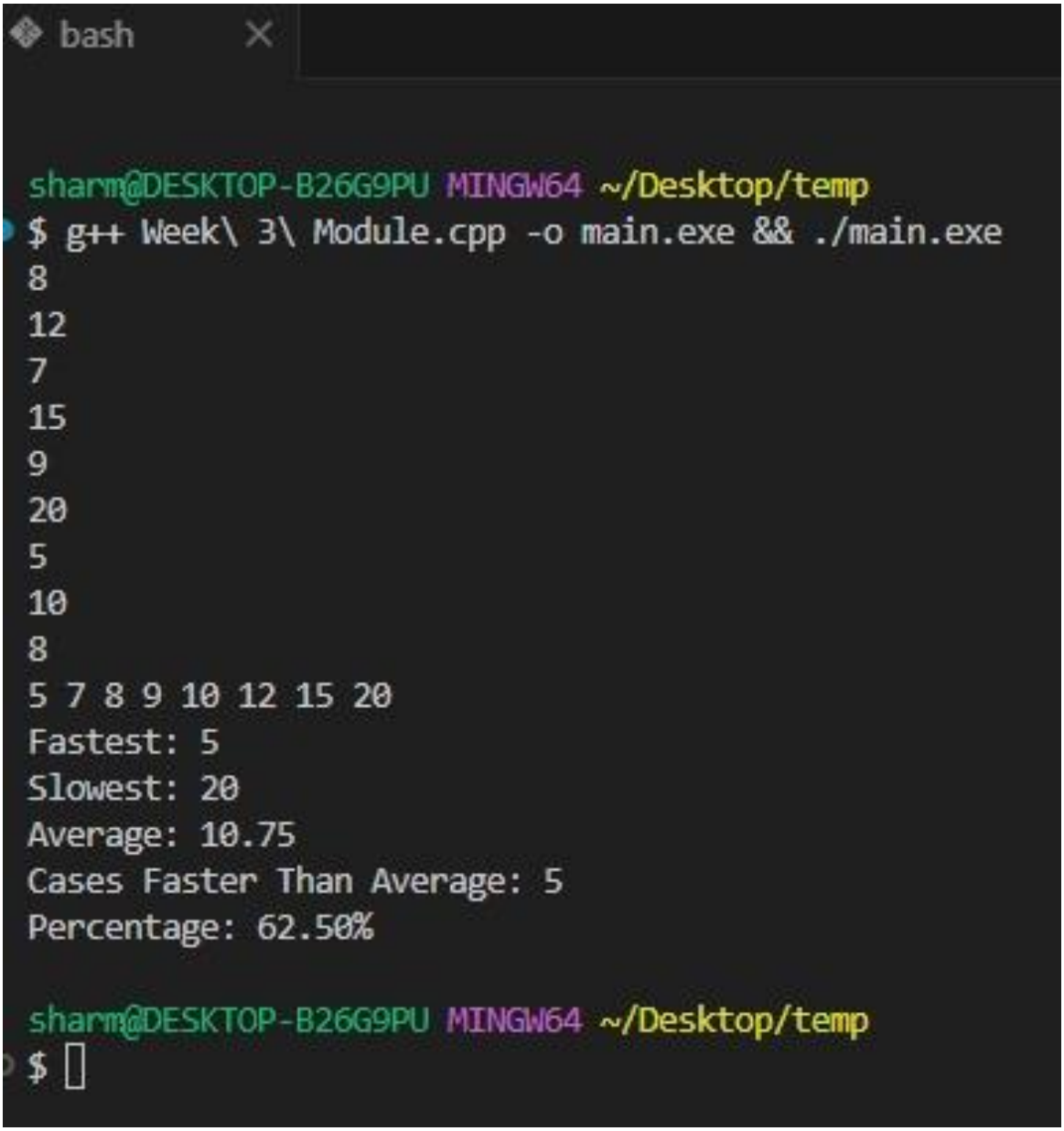
Question-2:

Week 3 Module.cpp

Week 3 Module.cpp > ...

```cpp
int main() {
    int N;
    cin >> N;

    vector<int> arr(N);
    int sum = 0;

    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        sum += arr[i];
    }

    heapSort(arr, N);

    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;

    cout << "Fastest: " << arr[0] << endl;

    cout << "Slowest: " << arr[N - 1] << endl;

    double average = (double)sum / N;
    cout << fixed << setprecision(2);
    cout << "Average: " << average << endl;

    int count = 0;
    for (int i = 0; i < N; i++) {
        if (arr[i] < average)
            count++;
    }

    cout << "Cases Faster Than Average: " << count << endl;
    double percentage = (double)count * 100 / N;
    cout << "Percentage: " << percentage << "%" << endl;

    return 0;
}
```

Result-2:


bash
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$ g++ Week\ 3\ Module.cpp -o main.exe && ./main.exe
8
12
7
15
9
20
5
10
8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
sharm@DESKTOP-B26G9PU MINGW64 ~/Desktop/temp
$